App Dev Project Report

Placement Portal Application (PPA)

1. Student Details

Name: Mridula J B

Roll Number: 23f2004413

Email: 23f2004413@ds.study.iitm.ac.in

About Me: I am a student at the IIT Madras BS Degree Program, currently pursuing my studies with a growing interest in web application development and building practical, real-world software solutions. I enjoy working on projects that combine backend logic, clean user interfaces, and thoughtful system design, and I'm always looking to strengthen my understanding of how full-stack applications come together end to end.

2. Project Details

Project Title: Placement Portal Application (PPA)

Problem Statement:

To design and build a web-based placement management system that streamlines the campus placement process by allowing companies to post placement drives, students to apply for suitable opportunities, and administrators to oversee and manage the entire process — replacing manual, fragmented coordination with a centralized digital platform.

Approach:

The application was built using Flask as the backend framework with a modular blueprint-based structure separating Admin, Company, and Student roles. Role-based JWT authentication secures access, SQLAlchemy models manage relational data (Users, Companies, Students, Drives, Applications), Redis is used for caching frequently accessed data, and Celery with Redis handles background jobs such as email reminders and report generation. The frontend is built with Vue.js (Options API) and Bootstrap 5 for a responsive, role-specific user interface.

3. AI/LLM Declaration

I used Claude (Anthropic) to assist in writing SQLAlchemy model definitions, resolving configuration issues (e.g., Celery setup), and improving code structure and documentation.

The extent of AI/LLM usage is around **20%**, limited to code suggestions, debugging guidance, and documentation formatting.

All final implementation logic, integration, and testing were done manually.

4. Technologies and Frameworks Used

Technology / Library / Tool	Purpose
Flask	Core backend web framework (acting as a REST API).
Vue.js 3 (Options API)	Client-side Single Page Application (SPA) framework for dynamic, reactive UI components.
Vite	Modern, high-performance frontend build tool and development server.
SQLAlchemy	Object-Relational Mapper (ORM) for SQL database mapping and query execution.

Technology / Library / Tool	Purpose
SQLite	Lightweight relational database for local persistent storage.
Redis	In-memory key-value store used for dashboard caching and as the Celery task broker.
Celery	Distributed task queue for running asynchronous jobs (CSV export) and periodic schedules (daily reminders, monthly reports).
Flask-JWT-Extended	Role-based token authentication and session security.
Axios	Promise-based HTTP client for making REST API calls from Vue to the Flask backend.
Bootstrap 5	Responsive layout grid, forms, buttons, tables, and modal components.
Flask-Mail	Composes and routes emails locally to MailHog.
MailHog	Local SMTP testing server that intercepts all outgoing emails and displays them in a web interface (http://localhost:8025).
Werkzeug	Provides secure password hashing (generate_password_hash) and safe file upload name formatting.

5. Database Schema / ER Diagram

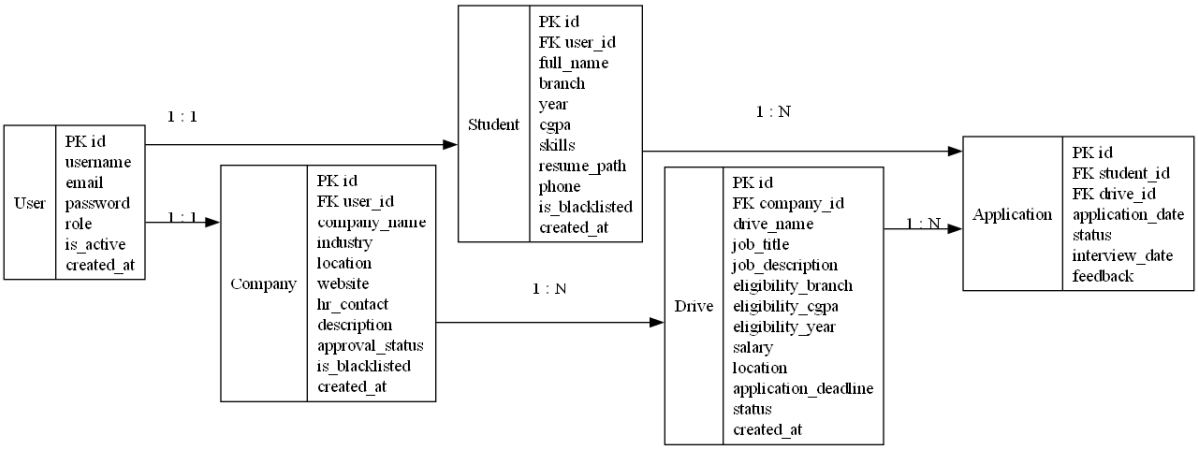
Tables:

- User — id (PK), username, email, password, role, is_active, created_at
- Student — id (PK), user_id (FK), full_name, branch, year, cgpa, skills, resume_path, phone, is_blacklisted, created_at
- Company — id (PK), user_id (FK), company_name, industry, location, website, hr_contact, description, approval_status, is_blacklisted, created_at
- Drive — id (PK), company_id (FK), drive_name, job_title, job_description, eligibility_branch, eligibility_cgpa, eligibility_year, salary, location, application_deadline, status, created_at
- Application — id (PK), student_id (FK), drive_id (FK), application_date, status, interview_date, feedback

Relationships:

- One-to-One → User → Student
- One-to-One → User → Company
- One-to-Many → Company → Drive
- One-to-Many → Student → Application
- One-to-Many → Drive → Application

ER Diagram:



6. API Resource Endpoints

Authentication (/api/auth)

Endpoint	Method	Description
/api/auth/register	POST	Register a new student or company profile.
/api/auth/login	POST	Authenticate user credentials and return a JWT access token.

Student Functionality (/api/student)

Endpoint	Method	Description
/api/student/dashboard	GET	Retrieve the list of active placement drives and student metrics (cached).
/api/student/profile	GET / PUT	View or update student profile details (branch, CGPA, skills, etc.).
/api/student/apply/<drive_id>	POST	Submit an application to a drive (validates eligibility and resume upload).
/api/student/applications	GET	View the list of submitted applications and candidate feedback.
/api/student/drives/search	GET	Search placement drives by job title.
/api/student/companies	GET	Fetch the list of approved, active companies.
/api/student/companies/<company_id>	GET	Fetch details of a company and its active drives.
/api/student/drives/<drive_id>	GET	Fetch requirements of a drive and check if already applied.
/api/student/upload-resume	POST	Upload a student resume in PDF format.
/api/student/resume/<student_id>	GET	Download a student's resume PDF (secured with IDOR protection).
/api/student/export-csv	POST	Trigger an asynchronous background task to export application history to CSV via email.

Recruiter Functionality (/api/company)

Endpoint	Method	Description
/api/company/dashboard	GET	Fetch placement drives posted by the authenticated company.
/api/company/profile	GET / PUT	View or update company profile details (overview, website, contact, etc.).
/api/company/drives	POST	Create a new placement drive (status set to pending approval).
/api/company/drives/<drive_id>/applications	GET	Fetch all candidate applications received for a specific drive.
/api/company/applications/<app_id>	PUT	Review an application and update status (Shortlist, Select, Reject) and feedback.
/api/company/drives/<drive_id>/close	PUT	Mark a placement drive as complete (status set to closed).

Administrator Functionality (/api/admin)

Endpoint	Method	Description
/api/admin/dashboard	GET	Fetch dashboard statistics (student, company, drive, and application counts).
/api/admin/companies	GET	Fetch the lists of pending signups and registered companies.
/api/admin/companies/<company_id>/approval	PUT	Approve or reject pending company registrations.
/api/admin/companies/<company_id>/blacklist	PUT	Toggle blacklist status of a company.
/api/admin/companies/<company_id>	DELETE	Delete a company profile and its associated user login credentials.
/api/admin/students	GET	Fetch the list of registered students.
/api/admin/students/<student_id>/blacklist	PUT	Toggle blacklist status of a student.
/api/admin/drives	GET	Fetch the lists of pending drive applications and ongoing drives.
/api/admin/drives/<drive_id>/approval	PUT	Approve, reject, or close a placement drive.
/api/admin/applications	GET	Fetch all candidate applications for administrative review.
/api/admin/reports	GET	Fetch KPIs (overall placement rate, status breakdown, and drive-wise metrics).

YAML API Definition File: Included separately in the submission ZIP as api.yaml.

7. Architecture and Features

Architecture Overview:

- app.py – Main Flask application entry point and database initialization.
- celery_config.py – Configures Celery settings and periodic Beat cron schedules.
- tasks.py – Defines asynchronous and periodic background tasks (deadline reminders, reports, CSV exports).
- cache.py – Handles connection setup and helper utilities for the Redis cache.
- /models – Database models using SQLAlchemy (User, Student, Company, Drive, Application).
- /routes – Flask Blueprints separating auth, student, company, and admin endpoints.
- /frontend – Client-side Single Page Application (SPA) structure built with Vite and Vue.js.

Implemented Features:

- Role-Based Authentication: Secure registration and login flow for Students, Companies, and Administrators using JSON Web Tokens (JWT).
- Direct Student Applications: Students can apply to active drives, update profiles, and upload resume PDFs.
- Recruiter Control Panel: Approved companies can post new drives, close completed drives, and review candidate submissions (shortlist, select, reject).
- Admin Verification Interface: Administrators can approve/reject company signups, blacklist/activate users, and approve/reject placement drives.
- Proactive Eligibility Enforcement: Automatic validation checks for minimum CGPA, branch, and graduation year criteria before accepting drive applications.
- In-Memory Caching & Performance: Redis cache integration to lower dashboard response times to sub-millisecond speeds.
- Asynchronous Jobs & Periodic Reminders: Celery tasks that send daily deadline notifications to students, email monthly HTML reports to the admin, and handle background CSV history exports.
- Secure Local Testing: Local SMTP MailHog configuration to intercept and test all outgoing emails offline.

8. Video Presentation

Drive Link: https://drive.google.com/file/d/1Z0bxtImcXWccdDx9a1NSUV6-qK8qCxdS/view?usp=sharing